Guion Técnico Maestro de Sustentación (10 Minutos) — Enfoque de Código y Arquitectura

QuickFood ERP • Plataforma Empresarial para Dark Kitchen

Seminario Desarrollo de Aplicaciones Web en Laravel con Enfoque RAD • COTECNOVA 2026

Entregable Segundo Corte — Avance del Proyecto (Video + Documento)

👥 Ficha Técnica y Distribución de Expositores

- **Asignatura:** Software de Gestión Empresarial
- **Docente Evaluador:** Comité Académico COTECNOVA
- **Expositor 1 (Estudiante 1): Miguel Ángel Ramírez Gutiérrez** (Arquitectura de Controladores, Modelos Eloquent, Base de Datos MySQL e Integración de Venta)
- **Expositor 2 (Estudiante 2): Andrés Felipe Mejía Suaza** (Servicio Transaccional `SaleService`, Form Requests, Seguridad RBAC Spatie y Suite de Pruebas PHPUnit)
- **Duración Estricta: 10:00 minutos exactos** *(Nota de rúbrica: lo que supere el minuto 10 no es evaluado)*
- **Enfoque de la Exposición: Técnico y de Código Fuente** (se explican clases, métodos, servicios, transacciones ACID, middlewares, bloqueos de concurrencia y consultas SQL en motor InnoDB).

🛠️ Preparación Técnica del Entorno (Antes de Iniciar la Grabación)

Para evitar pausas y optimizar el tiempo, se debe tener listo el entorno de trabajo:

1. **Editor de Código (Visual Studio Code):**
2. Pestaña 1: `app/Services/SaleService.php` (Zoom con `Ctrl + +` centrado en el método `register()`).
3. Pestaña 2: `app/Http/Controllers/SaleController.php` (métodos `store()` y `destroy()`).
4. Pestaña 3: `app/Http/Requests/StoreSaleRequest.php` (reglas de validación).
5. Pestaña 4: `database/seeders/RolePermissionSeeder.php` (matriz de permisos Spatie).
6. **Terminal Integrada o WSL Ubuntu (Fuente Grande 18px):**
7. Pestaña Terminal 1: Conectada al motor MySQL (`./vendor/bin/sail mysql` o `mysql -u root -p`).
8. Pestaña Terminal 2: Lista en la raíz del proyecto para ejecutar las pruebas: `php artisan test`.
9. **Navegador Web (Google Chrome o Edge):**
10. Pestaña 1: `http://localhost:8090/dashboard` (Panel principal con KPIs).
11. Pestaña 2: `http://localhost:8090/sales/create` (Formulario de nueva venta).
12. Pestaña 3: `http://localhost:8090/sales` (Histórico de ventas y anulación).
13. **Cronómetro de Referencia:** Celular visible para controlar el paso de cada bloque minuto a minuto.

⏱️ Matriz de Distribución de Tiempos y Roles

Minuto	Bloque Temático (Syllabus)	Expositor	Archivos de Código y Componentes Clave
0:00 – 1:00	**1. Presentación Institucional y Problemática**	Miguel (0:00-0:30) Andrés (0:30-1:00)	Arquitectura RAD Laravel 11, separación MVC + Services, problema de concurrencia en Dark Kitchen.

Minuto	Bloque Temático (Syllabus)	Expositor	Archivos de Código y Componentes Clave
1:00 – 2:30	**2. Modelo de Datos y Reglas de Negocio en Código**	Miguel (1:00-1:45) Andrés (1:45-2:30)	Migraciones `create_sales_table`, `create_sale_details_table`, relaciones Eloquent (`hasMany`, `belongsTo`), SoftDeletes y reglas de negocio.
2:30 – 6:30	**3. Demostración Funcional en Vivo desde el Código y BD**	Miguel (2:30-4:30) Andrés (4:30-6:30)	`StoreSaleRequest`, `SaleService::register()`, transacción InnoDB, bloqueo `lockForUpdate()`, queries SQL de stock, sobreventa (Rollback), anulación (`increment`), comprobante imprimible y KPIs en `DashboardController`.
6:30 – 8:30	**4. Seguridad, Roles Spatie y Middlewares**	Miguel (6:30-7:30) Andrés (7:30-8:30)	`RolePermissionSeeder.php`, `HasMiddleware`, directivas `@can` en Blade, demostración del error HTTP 403 `UnauthorizedException` y restauración con `SoftDeletes`.
8:30 – 9:30	**5. Inmersión en Código Transaccional (`SaleService`)**	Miguel (8:30-9:30)	Inspección línea a línea en VS Code de `DB::transaction()` y prevención de condiciones de carrera.
9:30 – 10:00	**6. Pruebas Automatizadas y Plan Tercer Corte**	Andrés (9:30-10:00)	Ejecución en terminal de `php artisan test` (22 tests en verde) y hoja de ruta tecnológica.

🎬 GUION TÉCNICO PASO A PASO (ACCIÓN Y CÓDIGO)

BLOQUE 1: Presentación, Empresa y Problema Arquitectónico

Tiempo: 0:00 – 1:00 (60 segundos)

- 💻 **Qué se muestra en pantalla:** Dashboard de QuickFood ERP y estructura del proyecto en VS Code destacando las carpetas `app/Http/Controllers`, `app/Services`, `app/Models` y `routes/web.php`.

🎙️ Qué dice MIGUEL ÁNGEL RAMÍREZ (0:00 – 0:30):

> *"Buenas noches docente y compañeros. Mi nombre es **Miguel Ángel Ramírez Gutiérrez**, y junto a mi compañero **Andrés Felipe Mejía Suaza**, presentamos la sustentación del Segundo Corte para la asignatura **Software de Gestión Empresarial** dentro del Seminario de Desarrollo de Aplicaciones Web en Laravel con Enfoque RAD de **COTECNOVA (2026)**.*
> *Nuestro proyecto se denomina **QuickFood ERP**, un sistema empresarial diseñado específicamente para optimizar el flujo operativo, transaccional y de inventario de una **Dark Kitchen (Cocina Oculta)** con modelo de pedidos y despachos a domicilio."*

🎙️ Qué dice ANDRÉS FELIPE MEJÍA (0:30 – 1:00):

> *"Desde el punto de vista arquitectónico, una cocina oculta opera con múltiples pedidos simultáneos en horarios pico. El reto central del negocio radicaba en la falta de integridad transaccional: se presentaban sobreventas de platos agotados y desincronización entre el área de despacho y la cocina.*
> *En este segundo corte, abordamos este problema mediante una arquitectura orientada a servicios en **Laravel 11**: implementamos transacciones atómicas con control de concurrencia en base de datos MySQL, un sistema de control de acceso basado en roles mediante la librería Spatie, y un panel de control con métricas en tiempo real calculadas mediante consultas optimizadas de agregación en Eloquent."*

BLOQUE 2: Modelo de Datos y Reglas de Negocio en el Código Fuente

Tiempo: 1:00 – 2:30 (90 segundos)

- 💻 **Qué se muestra en pantalla:** En pantalla completa el Diagrama Entidad-Relación extendido (`figura01_diagrama_er_extendido.png`) y el código de las migraciones en VS Code (`database/migrations/..._create_sales_table.php`).

🎙️ Qué dice MIGUEL ÁNGEL RAMÍREZ (1:00 – 1:45):

> *"Para este segundo corte, extendimos el esquema relacional mediante migraciones de Laravel, incorporando integridad referencial estricta:*
> *1. Creamos la tabla `clients` con campos para identificación tributaria y la bandera booleana `active`.*
> *2. Diseñamos la cabecera de facturación en la tabla `sales`, la cual se relaciona en esquema uno a muchos con la tabla `sale_details`.*
> *3. En las migraciones definimos claves foráneas con restricción estricta: `$table->foreignId('client_id')->constrained()->onDelete('restrict')`, impidiendo que se elimine un cliente que posea historial contable registrado.*
> *4. Cada detalle de venta vincula el campo foráneo `product_id` con la tabla `productos` y almacena campos inmutables como `quantity`, `unit_price` y `subtotal`.*
> *5. Implementamos además el trait `SoftDeletes` tanto en ventas como en productos y clientes, lo que genera la columna `deleted_at`, asegurando que ninguna eliminación física destruya la consistencia histórica de la contabilidad."*

🎙️ Qué dice ANDRÉS FELIPE MEJÍA (1:45 – 2:30):

> *"Para gobernar este modelo, implementamos en el backend tres reglas de negocio fundamentales a nivel de código:*
> * **Regla 1 (Aislamiento y Bloqueo Pesimista):** En la capa de servicios, cada venta se procesa dentro de una transacción gestionada por `DB::transaction()`, donde cada producto requerido se consulta con el método `Product::lockForUpdate()`. Esto aplica un bloqueo a nivel de fila en el motor InnoDB de MySQL para evitar colisiones de stock concurrentes.
> **Regla 2 (Validación Atómica y Rollback):** Si el stock actual es menor a la cantidad solicitada o si el producto tiene su estado inactivo, el servicio dispara una excepción `ValidationException::withMessages()`. Al lanzarse la excepción, Laravel efectúa un Rollback automático en la base de datos sin alterar ningún registro.
> **Regla 3 (Devolución de Inventario en Anulación):** Cuando se anula una venta en `SaleController@destroy`, se ejecuta una transacción que itera sobre los detalles e invoca el método atómico `$detail->product->increment('stock', $detail->quantity)`, actualizando el estado de la factura a 'anulada' antes de ejecutar el borrado lógico."*

BLOQUE 3: Demostración Funcional en Vivo desde el Código y Base de Datos

Tiempo: 2:30 – 6:30 (4 minutos)

- 💻 **Qué se muestra en pantalla:** Pantalla dividida o alternancia rápida: Navegador Web + Terminal con MySQL interactivo + Métodos en VS Code.

🎙️ Qué dice MIGUEL ÁNGEL RAMÍREZ (2:30 – 3:30) — [Paso a Paso: Estado Inicial y Validación Form Request]:

- **Acción en pantalla:** Abrir la terminal con MySQL y consultar el producto ID 1:
  `sql SELECT id, nombre, stock FROM productos WHERE id = 1;`
  (Se observa en la salida de consola: Stock = 50 unidades).
- **Luego cambiar a VS Code** y señalar el archivo `app/Http/Requests/StoreSaleRequest.php`.

> *"Para iniciar la demostración técnica, consultamos directamente en el motor MySQL el estado del inventario: el producto ID 1, 'Hamburguesa Monster Doble', cuenta con exactamente **50 unidades** disponibles en la base de datos. Al momento de enviar el formulario de venta en el ERP, la petición HTTP POST es interceptada por el Form Request `StoreSaleRequest`.*
> *Observemos cómo este archivo valida:*
> *Primero, que el cliente exista en la base de datos y no esté eliminado lógicamente mediante `Rule::exists('clients', 'id')->whereNull('deleted_at')->where('active', true)`.*
> *Segundo, valida la matriz de productos asegurando que el array `items` contenga al menos un elemento, que cada `product_id` sea válido y que el campo `quantity` sea un entero positivo mayor o igual a 1."*

🎙️ Qué dice ANDRÉS FELIPE MEJÍA (3:30 – 4:30) — [Paso a Paso: Transacción, Descuento en MySQL e Impuestos]:

- **Acción en pantalla:** En el navegador (`/sales/create`), registrar una venta de **2 unidades** del producto ID 1. Presionar "Guardar Venta".
- **Luego ir de inmediato a la terminal de MySQL** y volver a correr:
  `sql SELECT id, nombre, stock FROM productos WHERE id = 1;`
  (Se observa en la consola que el stock bajó a 48 unidades).

> *"Una vez validado el request, el controlador invoca a `SaleService::register()`.*
> *En este punto, el código calcula el impuesto tributario del 19% mediante la instrucción `$tax = round($subtotal * 0.19, 2)`, calcula el total general y genera el número consecutivo secuencial formateado con padding como `'FV-00001'`.*
> *Dentro del mismo closure de la transacción, Laravel ejecuta `$item['product']->decrement('stock', $item['quantity'])`.*
> *Si regresamos a la terminal de MySQL y volvemos a consultar la tabla `productos`, evidenciamos que el stock disminuyó en caliente de 50 a **48 unidades**, certificando que el descuento fue ejecutado de forma atómica en el motor de persistencia."*

🎙️ Qué dice MIGUEL ÁNGEL RAMÍREZ (4:30 – 5:30) — [Paso a Paso: Provocación de Error por Stock Insuficiente y Rollback]:

- **Acción en pantalla:** En el formulario de ventas, intentar vender **999 unidades** del producto.
- **Presionar "Guardar Venta":** El sistema rechaza la petición y muestra el mensaje de error: "*Stock insuficiente para «Hamburguesa Monster Doble». Disponible: 48, solicitado: 999.*"

- **Mostrar terminal MySQL:** Ejecutar de nuevo el SELECT:
  `sql SELECT id, nombre, stock FROM productos WHERE id = 1;`
  (El stock permanece intacto en 48 unidades).

> *"Ahora provocamos intencionalmente la condición de fallo para validar la regla de concurrencia: intentamos facturar 999 unidades, una cantidad que supera las existencias.*
> *En el código de `SaleService`, la condición `if ($product->stock < $quantity)` se evalúa verdadera y dispara una excepción de tipo `ValidationException`.*
> *Debido a que esta excepción ocurre dentro del closure de `DB::transaction()`, el motor de base de datos ejecuta un **Rollback inmediato**: no se crea la factura en `sales`, no se insertan registros en `sale_details` y el inventario en MySQL se mantiene inalterado en 48 unidades.*
> *Esto previene inconsistencias y saldos negativos en el sistema contable."*

🎙️ Qué dice ANDRÉS FELIPE MEJÍA (5:30 – 6:30) — [Paso a Paso: Anulación, Restitución de Stock, Factura PDF y Dashboard]:

- **Acción en pantalla:**
- En el listado de ventas (`/sales`), abrir la venta generada (`FV-00001`) mostrando la vista de comprobante electrónico (`/sales/show.blade.php`), diseñada con CSS `@media print` para exportación a PDF.
- Presionar el botón "Anular Venta".
- Consultar la terminal de MySQL:
  `sql SELECT id, nombre, stock FROM productos WHERE id = 1;`
  (El stock regresa exactamente a 50 unidades).
- Ir al Dashboard (`/dashboard`) y señalar los KPIs actualizados.

> *"En la vista de detalle de la venta, el sistema renderiza el comprobante fiscal estructurado con resolución DIAN simulada, desglose de base gravable e IVA del 19%, apto para impresión y descarga en PDF gracias a los estilos `@media print`.*
> *A continuación, ejecutamos la anulación de la factura. En el código, el método `SaleController@destroy` ejecuta una transacción que invoca `$detail->product->increment('stock', $detail->quantity)`, actualiza el estado a 'anulada' y aplica `SoftDeletes`.*
> *Si verificamos nuevamente en MySQL, comprobamos que el stock fue reintegrado exitosamente a sus **50 unidades originales**.*
> *Finalmente, al consultar el panel del Dashboard, el método `DashboardController@index` ejecuta agregaciones en tiempo real: calcula el valor patrimonial del inventario mediante `sum(DB::raw('price * stock'))` y actualiza el **Top 5** de productos con `groupBy('product_id')->orderByDesc('total_sold')`."*

BLOQUE 4: Seguridad, Roles Spatie y Middlewares en el Backend

Tiempo: 6:30 – 8:30 (2 minutos)

- 💻 **Qué se muestra en pantalla:** Código en VS Code de `RolePermissionSeeder.php`, `SaleController.php` (método `middleware()`), directivas `@can` en `layouts/app.blade.php`, e intento de acceso no autorizado en el navegador.

🎙️ Qué dice MIGUEL ÁNGEL RAMÍREZ (6:30 – 7:30):

- **Acción en pantalla:** Abrir en VS Code `database/seeders/RolePermissionSeeder.php` y luego `app/Http/Controllers/SaleController.php`.

> *"La seguridad del ERP está cimentada sobre el paquete oficial `spatie/laravel-permission`.*
> *En `RolePermissionSeeder.php` definimos los roles organizacionales: **Administrador**, **Vendedor** y **Almacenista**.*
> *Cada rol recibe permisos granulares por módulo mediante `$role->syncPermissions(...)`: por ejemplo, `ver-ventas`, `crear-ventas`, `ver-productos` y `ver-papelera`.*
> *A nivel de backend, protegemos los controladores implementando la interfaz `HasMiddleware` de Laravel 11. En `SaleController`, el método estático `middleware()` define:*
> `new Middleware('permission:ver-ventas', only: ['index', 'show'])`
> `new Middleware('permission:crear-ventas', only: ['create', 'store'])`
> `new Middleware('permission:eliminar-ventas', only: ['destroy'])`.
> *Esto asegura que ninguna petición HTTP no autorizada llegue al código de ejecución, independientemente de la interfaz."*

🎙️ Qué dice ANDRÉS FELIPE MEJÍA (7:30 – 8:30):

- **Acción en pantalla:**
- Mostrar en VS Code `resources/views/layouts/app.blade.php` con las directivas `@can('ver-ventas')`.
- En el navegador, cambiar rápidamente de usuario al rol **Almacenista** (`almacenista@test.com`) mediante el conmutador de autenticación.
- En la barra de direcciones del navegador, intentar escribir manualmente la URL: `http://localhost:8090/sales`.
- El servidor aborta la petición arrojando un error **HTTP 403 | USER DOES NOT HAVE THE RIGHT PERMISSIONS**.
- Volver al usuario Administrador y abrir el módulo de Papelera (`/productos?papelera=1`), mostrando la restauración con `Product::withTrashed()->restore()`.

> *"En la capa de presentación (Blade), las opciones del menú están condicionadas mediante directivas `@can('ver-ventas')` y `@can('crear-pedidos')`, ocultando los elementos a usuarios sin los privilegios requeridos.*
> *Para demostrar la protección estricta en el servidor, iniciamos sesión como 'Almacenista' —usuario que carece del permiso `ver-ventas`— e intentamos forzar el acceso por URL escribiendo `/sales`.*
> *De inmediato, el middleware de Spatie intercepta la solicitud, comprueba que la matriz de permisos del usuario no contiene la facultad requerida y aborta la ejecución lanzando una excepción `UnauthorizedException`, retornando el código de respuesta HTTP 403 Forbidden.*
> *Asimismo, el rol Administrador dispone del módulo de Papelera con SoftDeletes, permitiendo consultar registros marcados con `deleted_at IS NOT NULL` y restaurarlos formalmente mediante el método `restore()` sin pérdida de integridad referencial."*

BLOQUE 5: Inmersión Técnica en el Código Transaccional (`SaleService`)

Tiempo: 8:30 – 9:30 (1 minuto)

- 💻 **Qué se muestra en pantalla:** VS Code en pantalla completa en el archivo `app/Services/SaleService.php`, haciendo zoom en el bloque transaccional entre las líneas 18 y 85.

🎙️ Qué dice MIGUEL ÁNGEL RAMÍREZ (8:30 – 9:30):

- **Acción en pantalla:** Seleccionar con el cursor las líneas de `DB::transaction()` y `lockForUpdate()`.

> *"En este bloque destacamos el corazón transaccional del ERP: la clase de servicio `SaleService.php`.*
> *Separamos la lógica de negocio del controlador para garantizar alta cohesión y bajo acoplamiento:*
> *1. El método `register()` encapsula toda la operación dentro de un `DB::transaction()`. Esto garantiza el cumplimiento estricto de las propiedades **ACID** de las bases de datos relacionales:*
> * **Atomicidad:** O se crea la cabecera, los detalles y se descuenta el stock de todos los productos, o no se efectúa absolutamente nada.
> **Aislamiento:** La instrucción `Product::lockForUpdate()->findOrFail($productId)` ejecuta un bloqueo pesimista a nivel de fila (`SELECT ... FOR UPDATE`). Si otro proceso concurrente intenta consultar el mismo producto antes de que este commit finalice, queda en espera encolada, evitando condiciones de carrera (Race Conditions) y lecturas sucias.
> 2. El servicio consolida además cantidades si un mismo ítem se envía duplicado en la petición, calcula el impuesto del 19% de forma inmutable y utiliza `Sale::withTrashed()->orderByDesc('id')->first()` para generar un consecutivo contable unívoco sin saltos numéricos."*

BLOQUE 6: Pruebas Automatizadas en Verde y Proyección del Tercer Corte

Tiempo: 9:30 – 10:00 (30 segundos)

- 💻 **Qué se muestra en pantalla:** Terminal de comandos ejecutando las pruebas con `php artisan test`, mostrando los 22 tests en color verde pasando exitosamente.

🎙️ Qué dice ANDRÉS FELIPE MEJÍA (9:30 – 10:00):

- **Acción en pantalla:** Ejecutar en la terminal:
  `bash php artisan test`
  (Se muestran los tests de `Clase9IntegracionModulosTest` y `Clase7RolesPermisosTest` con `PASS` y `OK (22 tests, 58 assertions)`).

> *"Para certificar que toda la lógica descrita es robusta y tolerante a fallos, construimos una batería de 22 pruebas automatizadas en PHPUnit:*
> *Como observamos en consola, la ejecución de `php artisan test` arroja un resultado del **100% de tests en verde**: validamos transacciones completas, rollbacks por falta de stock, restricciones de claves foráneas en clientes, respuesta 403 por permisos y cálculo de KPIs.*
> *Para el Tercer Corte final, implementaremos:*
> *1. Módulo de auditoría y log de eventos críticos.*
> *2. Una API RESTful con autenticación Sanctum para la sincronización con la aplicación móvil de los repartidores.*
> *3. Colas de trabajo (Queues) y automatización con Laravel Scheduler para el despacho asíncrono de facturas electrónicas por correo.*
> *Muchas gracias por su atención."*

📋 Resumen Rápido de Comandos para los Expositores

```
# 1. Conexión a la base de datos MySQL para evidenciar el stock:
./vendor/bin/sail mysql -u root -p
USE quickfood_erp;
SELECT id, nombre, stock FROM productos WHERE id = 1;

# 2. Ejecutar la suite completa de pruebas automatizadas:
php artisan test --testsuite=Feature

# 3. Levantar los contenedores de Docker (si estuviesen apagados):
./vendor/bin/sail up -d
```

Documento compilado y optimizado según los lineamientos oficiales de COTECNOVA (Seminario RAD Laravel 2026).
Autores: Miguel Ángel Ramírez Gutiérrez & Andrés Felipe Mejía Suaza.